



Proyecto Final

Simulación de Servidor MMO

Adolfo Iván Pérez Núñez - 181916

Patricio Funes San Martín - 181491

Sebastian Salazar Rodríguez - 183803

Sistemas Operativos

Profesor: Edwin Montes Orozco

Universidad de las Américas Puebla

Primavera 2026

Índice

1. Descripción del proyecto	2
2. Diseño y arquitectura	2
3. Módulos implementados	4
4. Experimentos realizados	6
5. Resultados	8
6. Interpretación técnica	14
7. Limitaciones y trabajo futuro	15

1. Descripción del proyecto

El proyecto consiste en la simulación de un servidor para un videojuego multijugador masivo en línea, conocido como MMO. En este sistema, varios jugadores intentan conectarse al mismo servidor, moverse dentro de un mapa compartido, recoger objetos limitados y enviar mensajes a un chat global.

La finalidad del proyecto es representar un sistema concurrente real, donde múltiples hilos compiten por recursos compartidos. Los recursos principales del sistema son el mapa del mundo, los objetos disponibles en el suelo, el cupo del servidor y el buffer de chat.

El programa fue desarrollado en lenguaje C para ejecutarse en un entorno Linux. Para manejar la concurrencia se utiliza la biblioteca `pthread`, junto con metodos de sincronización como mutex, semáforos y variables de condición.

La simulación permite observar conceptos vistos en clase, como la creación de hilos, la espera por recursos, la entrada a secciones críticas, la exclusión mutua y la sincronización entre hilos.

Elegimos un servidor de videojuego porque un servidor en línea necesita coordinar muchas acciones simultáneas. Si dos jugadores intentan recoger el mismo objeto o moverse a la misma posición al mismo tiempo, el servidor debe controlar el acceso para evitar inconsistencias en el estado del mundo virtual.

2. Diseño y arquitectura

El sistema está organizado en varios archivos de código fuente. Cada archivo cumple una función específica dentro de la simulación.

Archivo	Función principal
<code>main.c</code>	Punto de entrada del programa. Inicializa el mundo, crea los hilos, controla el tiempo de simulación, realiza el apagado limpio y muestra el resumen final.
<code>mmo_server.h</code>	Contiene las constantes, estructuras de datos, recursos compartidos y prototipos de funciones usados por todo el proyecto.
<code>mundo.c</code>	Inicializa el mapa, los jugadores, los ítems, los mutex, el semáforo, la variable de condición y las funciones de impresión en consola.

<code>jugador.c</code>	Define el comportamiento de cada jugador: conexión al servidor, movimiento en el mapa, recolección de ítems, envío de mensajes y desconexión.
<code>entorno.c</code>	Implementa los hilos de entorno, incluyendo el respawn de objetos, el procesamiento del chat y el game loop del servidor.

La estructura principal del sistema es `MundoMM0`. Esta estructura concentra el estado global de la simulación. Dentro de ella se encuentran el mapa, el arreglo de ítems, el buffer de chat, el semáforo de cupo, los jugadores, los mutex y las estadísticas.

El mapa se representa como una matriz de 10x10 casillas. Cada casilla puede estar vacía, contener un jugador o contener un ítem. Para proteger el acceso concurrente al mapa, cada casilla tiene su propio mutex. Esto permite que jugadores ubicados en zonas distintas puedan moverse al mismo tiempo sin bloquear todo el mapa.

Los ítems se almacenan en un arreglo global. Cada ítem tiene coordenadas, nombre, estado activo y un mutex individual. Esto permite controlar la recolección de objetos limitados cuando varios jugadores intentan recoger el mismo recurso.

El chat se implementa mediante un buffer circular compartido. Los jugadores escriben mensajes en el buffer y un hilo especializado los procesa. Para sincronizar este comportamiento se utiliza un mutex de chat y una variable de condición.

El cupo del servidor se controla mediante un semáforo contador. En la configuración del proyecto, el servidor permite un máximo de 5 jugadores conectados, mientras que 8 jugadores intentan conectarse. Esto permite observar que algunos hilos deben esperar cuando el servidor está lleno.

El flujo general del sistema es el siguiente:

1. El programa inicializa el mundo y las primitivas de sincronización.
2. Se crean los hilos de jugadores, entorno, chat y game loop.
3. Los jugadores intentan conectarse al servidor mediante el semáforo.
4. Los jugadores conectados se mueven, recogen ítems y envían mensajes.
5. El hilo de entorno genera nuevos ítems periódicamente.
6. El hilo de chat procesa mensajes cuando recibe una señal.
7. Al finalizar el tiempo de simulación, el sistema apaga los hilos y destruye las primitivas.

3. Módulos implementados

El proyecto implementa varios módulos funcionales que permiten demostrar conceptos de concurrencia y sincronización.

Módulo de conexión de jugadores

Este módulo controla el acceso de jugadores al servidor. Cada jugador ejecuta `sem_wait()` para intentar entrar. Si hay cupo disponible, el jugador se conecta. Si el servidor está lleno, el hilo queda bloqueado hasta que otro jugador se desconecte y libere un espacio con `sem_post()`.

Ejemplo de salida:

```
[JUGADOR 06] SERVIDOR LLENO - Entrando a sala de espera
[JUGADOR 06] sem_wait() exitoso - CONECTADO
[JUGADOR 02] sem_post() - DESCONECTADO
```

Módulo de movimiento en el mapa

Este módulo permite que los jugadores se muevan a casillas adyacentes dentro del mapa. Antes de modificar la matriz compartida, el jugador bloquea los mutex de las casillas involucradas. Si la casilla destino está vacía o contiene un ítem, el movimiento se realiza. Si está ocupada, el movimiento se cancela.

Ejemplo de salida:

```
[JUGADOR 03] Movimiento derecha (2,4)->(3,4) - Solicitando Mutex casilla destino...
[JUGADOR 03] Mutex casilla adquirido - Verificando casilla (3,4)...
[JUGADOR 03] Posicion actualizada (2,4)->(3,4) - Liberando Mutex.
```

Módulo de recolección de ítems

Este módulo controla la competencia por objetos limitados. Cuando un jugador llega a una casilla con un ítem, intenta recogerlo. Para hacerlo, bloquea el mutex individual del objeto y valida si el ítem sigue activo.

La segunda validación dentro del mutex es importante porque otro jugador pudo haber recogido el mismo objeto mientras el primer jugador esperaba acceso a la sección crítica.

Ejemplo de salida:

```
[JUGADOR 04] Mutex del item adquirido - Validando existencia de 'Gema'...  
[JUGADOR 04] ** ITEM RECOGIDO ** 'Gema'  
[JUGADOR 07] ** RACE CONDITION EVITADA ** 'Gema' ya fue recogido por otro jugador.
```

Módulo de chat global

Este módulo permite que los jugadores envíen mensajes a un buffer compartido. El hilo de chat permanece dormido usando `pthread_cond_wait()` mientras no hay mensajes. Cuando un jugador escribe un mensaje, se ejecuta `pthread_cond_signal()` para despertar al hilo de broadcast.

Ejemplo de salida:

```
[CHAT] pthread_cond_wait() - Hilo dormido, esperando signal...  
[JUGADOR 01] Mensaje enviado - Liberando Mutex de chat. Signal enviado.  
[CHAT] Variable de condicion: DESPIERTO - Signal recibido!  
[CHAT] Procesando mensaje del Jugador 01
```

Módulo de respawn de ítems

Este módulo representa un hilo de entorno que trabaja en segundo plano. Cada cierto intervalo de tiempo, el hilo intenta colocar un nuevo ítem en una casilla vacía del mapa. Para hacerlo, bloquea el mutex de la casilla seleccionada y valida que esté libre antes de modificarla.

Ejemplo de salida:

```
[WORLD] Tiempo de respawn alcanzado - Calculando coordenadas...  
[WORLD] Intentando casilla (5,7) - Esperando Mutex casilla...  
[WORLD] Mutex adquirido - 'Pocion+' colocado en (5,7) - Liberando Mutex.
```

Módulo de game loop

Este módulo imprime periódicamente el estado del servidor. Muestra información como jugadores conectados, ítems activos, movimientos realizados, mensajes enviados y condiciones de carrera controladas. También imprime una representación del mapa en consola.

Ejemplo de representación del mapa en consola:

```

=== MAPA DEL MUNDO ===
  0 1 2 3 4 5 6 7 8 9
0 . . . . .
1 . . * . . . . P .
2 . . . . .
3 . P . . . * . . .
4 . . . . .
5 . . . . P . . . .
6 . . * . . . .
7 . . . . . P . .
8 . . . . . . . *
9 . . . . .

```

4. Experimentos realizados

Experimento 1: Control de cupo con semáforo

Objetivo: observar cómo el servidor controla el número máximo de jugadores conectados.

Configuración:

- MAX_PLAYERS = 5
- TOTAL_PLAYERS = 8

Procedimiento: se ejecuta la simulación con más jugadores intentando conectarse que espacios disponibles en el servidor.

Resultado esperado: solo 5 jugadores pueden estar conectados al mismo tiempo. Los jugadores restantes deben esperar hasta que se libere un espacio.

Concepto demostrado: uso de semáforo contador para administrar un recurso limitado.

Experimento 2: Movimiento concurrente en el mapa

Objetivo: observar cómo varios jugadores modifican una matriz compartida sin generar posiciones inconsistentes.

Procedimiento: los jugadores conectados se mueven de forma aleatoria por el mapa. Cada movimiento requiere bloquear las casillas involucradas.

Resultado esperado: los jugadores cambian de posición sin superponerse en la misma casilla. Si una casilla destino está ocupada, el movimiento se cancela.

Concepto demostrado: exclusión mutua sobre una estructura compartida.

Experimento 3: Recolección de objetos limitados

Objetivo: demostrar qué ocurre cuando varios hilos compiten por un mismo recurso limitado.

Procedimiento: los jugadores se mueven por el mapa y, si llegan a una casilla con un ítem, intentan recogerlo.

Resultado esperado: solo un jugador puede recoger cada ítem. Si otro jugador intenta recoger el mismo objeto después, el sistema detecta que el ítem ya no está activo.

Concepto demostrado: control de condiciones de carrera mediante mutex y doble validación.

Experimento 4: Chat con variable de condición

Objetivo: mostrar la coordinación entre hilos productores y un hilo consumidor.

Procedimiento: los jugadores escriben mensajes en el buffer de chat. El hilo de broadcast procesa los mensajes cuando recibe una señal.

Resultado esperado: el hilo de chat permanece dormido mientras no hay mensajes y despierta cuando un jugador envía uno.

Concepto demostrado: uso de variable de condición para evitar espera activa.

Experimento 5: Respawn de ítems

Objetivo: observar cómo un hilo de entorno modifica el mapa al mismo tiempo que los jugadores interactúan con él.

Procedimiento: el hilo de mundo espera un intervalo de tiempo y después intenta colocar un nuevo ítem en una casilla vacía.

Resultado esperado: los nuevos ítems aparecen en el mapa sin sobrescribir jugadores ni otros ítems.

Concepto demostrado: sincronización entre hilos de usuario simulados y un hilo de entorno.

5. Resultados

Durante la ejecución, el sistema muestra logs en consola con diferentes etiquetas para identificar el origen de cada acción:

- [SERVER]: eventos generales del servidor.
- [JUGADOR XX]: acciones realizadas por cada jugador.
- [WORLD]: acciones del hilo de entorno.
- [CHAT]: procesamiento de mensajes del chat.

Estos logs permiten observar qué hilo está ejecutando una acción, qué recurso intenta usar, cuándo espera, cuándo entra a una sección crítica y cuándo libera el recurso. Para documentar la evidencia, se tomaron capturas de los momentos más importantes de la simulación.

Compilación y ejecución

Los comandos usados para compilar y ejecutar el proyecto fueron los siguientes:

```
gcc -Wall -Wextra -std=gnu11 -o mmo_server main.c mundo.c jugador.c entorno.c -  
lpthread
```

```
./mmo_server
```

Inicio de la simulación

Al iniciar la simulación, el programa muestra la configuración general del servidor: cupo máximo de jugadores, jugadores totales, tamaño del mapa, cantidad de ítems iniciales y duración de la simulación. También se imprime el mapa inicial, donde el símbolo . representa una casilla vacía y el símbolo * representa un ítem disponible.

```
pato@pato-VirtualBox:~/Downloads$ ./mmo_server

SIMULADOR DE SERVIDOR MMO - SISTEMAS OPERATIVOS
UDLAP - Primavera 2026

Cupo maximo: 5 jugadores
Jugadores totales: 8 (algunos esperaran)
Mapa: 10x10 casillas
Items iniciales: 6
Duracion: 12 segundos

[SERVER] Fase 1: Inicializando estructuras y primitivas...
[SERVER] Mundo creado. Mapa 10x10 con 6 items.
[SERVER] Semaforo de cupo inicializado con valor 5.

=== MAPA DEL MUNDO ===
  0 1 2 3 4 5 6 7 8 9
0 . . . . . . . . . .
1 . . . . . . . * .
2 . . . . . . . . .
3 . . . . . * . . .
4 . . . . . . . . .
5 . * . . . . . . .
6 . . . . . . . * .
7 . . . . . . . . .
8 . . . . * . . . .
9 . . . . . * . . .
```

Figura 1: Inicio de la simulación del servidor MMO. Se muestra la configuración inicial del sistema, el cupo máximo de jugadores, los ítems iniciales y el mapa compartido.

Control de cupo con semáforo

Uno de los resultados principales es el control del cupo máximo del servidor. En la simulación, existen más jugadores intentando conectarse que espacios disponibles. Por esta razón, algunos hilos entran en espera cuando el servidor está lleno. Esto se observa mediante los mensajes `sem_wait()`, `SERVIDOR LLENO` y `sem_post()`.

```

36:[JUGADOR 02] Solicita conexion al servidor - sem_wait() en cupo...
37:[JUGADOR 02] sem_wait() exitoso - CONECTADO (Cupo: 1/5)
42:[JUGADOR 03] Solicita conexion al servidor - sem_wait() en cupo...
43:[JUGADOR 03] sem_wait() exitoso - CONECTADO (Cupo: 2/5)
55:[JUGADOR 05] Solicita conexion al servidor - sem_wait() en cupo...
56:[JUGADOR 05] sem_wait() exitoso - CONECTADO (Cupo: 3/5)
65:[JUGADOR 06] Solicita conexion al servidor - sem_wait() en cupo...
66:[JUGADOR 06] sem_wait() exitoso - CONECTADO (Cupo: 4/5)
71:[JUGADOR 07] Solicita conexion al servidor - sem_wait() en cupo...
72:[JUGADOR 07] sem_wait() exitoso - CONECTADO (Cupo: 5/5)
89:[JUGADOR 08] Solicita conexion al servidor - sem_wait() en cupo...
90:[JUGADOR 08] SERVIDOR LLENO - Entrando a sala de espera (bloqueado en sem_wait)...
101:[JUGADOR 01] Solicita conexion al servidor - sem_wait() en cupo...
102:[JUGADOR 01] SERVIDOR LLENO - Entrando a sala de espera (bloqueado en sem_wait)...
109:[JUGADOR 04] Solicita conexion al servidor - sem_wait() en cupo...
110:[JUGADOR 04] SERVIDOR LLENO - Entrando a sala de espera (bloqueado en sem_wait)...
425:[JUGADOR 03] sem_post() - DESCONECTADO (Cupo: 4/5) Items recogidos: 0
426:[JUGADOR 08] sem_wait() exitoso - CONECTADO (Cupo: 5/5)
435:[JUGADOR 02] sem_post() - DESCONECTADO (Cupo: 4/5) Items recogidos: 0
436:[JUGADOR 01] sem_wait() exitoso - CONECTADO (Cupo: 5/5)
459:[JUGADOR 05] sem_post() - DESCONECTADO (Cupo: 4/5) Items recogidos: 1
460:[JUGADOR 04] sem_wait() exitoso - CONECTADO (Cupo: 5/5)
483:[JUGADOR 07] sem_post() - DESCONECTADO (Cupo: 4/5) Items recogidos: 1
545:[JUGADOR 06] sem_post() - DESCONECTADO (Cupo: 3/5) Items recogidos: 0
557:[JUGADOR 04] sem_post() - DESCONECTADO (Cupo: 2/5) Items recogidos: 0
559:[JUGADOR 08] sem_post() - DESCONECTADO (Cupo: 1/5) Items recogidos: 0
561:[JUGADOR 01] sem_post() - DESCONECTADO (Cupo: 0/5) Items recogidos: 0

```

Figura 2: Control de cupo mediante semáforo contador. Los jugadores ejecutan `sem_wait()` para conectarse y, cuando el servidor está lleno, algunos hilos quedan bloqueados hasta que otro jugador libera espacio con `sem_post()`.

Movimiento concurrente en el mapa

Durante la simulación, los jugadores se mueven dentro del mapa compartido. Antes de actualizar su posición, cada jugador solicita el mutex de la casilla correspondiente. En los logs se puede observar cuándo un jugador solicita el mutex, cuándo lo adquiere, cuándo actualiza su posición y cuándo libera el recurso. También se muestran casos donde el movimiento se cancela porque la casilla destino está ocupada.

```

39:[JUGADOR 07] Movimiento abajo (5,6)->(5,7) - Solicitando Mutex casilla destino...
40:[JUGADOR 07] Mutex casilla adquirido - Verificando casilla (5,7)...
41:[JUGADOR 07] Posicion actualizada (5,6)->(5,7) - Liberando Mutex.
42:[JUGADOR 07] Movimiento derecha (5,7)->(6,7) - Solicitando Mutex casilla destino...
43:[JUGADOR 07] Mutex casilla adquirido - Verificando casilla (6,7)...
44:[JUGADOR 07] Posicion actualizada (5,7)->(6,7) - Liberando Mutex.
48:[JUGADOR 06] Movimiento derecha (3,9)->(4,9) - Solicitando Mutex casilla destino...
49:[JUGADOR 06] Mutex casilla adquirido - Verificando casilla (4,9)...
50:[JUGADOR 06] Posicion actualizada (3,9)->(4,9) - Liberando Mutex.
54:[JUGADOR 08] Movimiento derecha (4,7)->(5,7) - Solicitando Mutex casilla destino...
55:[JUGADOR 08] Mutex casilla adquirido - Verificando casilla (5,7)...
56:[JUGADOR 08] Posicion actualizada (4,7)->(5,7) - Liberando Mutex.
57:[JUGADOR 06] Movimiento arriba (4,9)->(4,8) - Solicitando Mutex casilla destino...
58:[JUGADOR 06] Mutex casilla adquirido - Verificando casilla (4,8)...
59:[JUGADOR 06] Posicion actualizada (4,9)->(4,8) - Liberando Mutex.
63:[JUGADOR 03] Movimiento abajo (1,0)->(1,1) - Solicitando Mutex casilla destino...
64:[JUGADOR 03] Mutex casilla adquirido - Verificando casilla (1,1)...
65:[JUGADOR 03] Posicion actualizada (1,0)->(1,1) - Liberando Mutex.
76:[JUGADOR 08] Movimiento derecha (5,7)->(6,7) - Solicitando Mutex casilla destino...
77:[JUGADOR 08] Mutex casilla adquirido - Verificando casilla (6,7)...
78:[JUGADOR 08] Casilla (6,7) ocupada - Liberando Mutex. Movimiento cancelado.
79:[JUGADOR 07] Movimiento abajo (6,7)->(6,8) - Solicitando Mutex casilla destino...
80:[JUGADOR 07] Mutex casilla adquirido - Verificando casilla (6,8)...
81:[JUGADOR 07] Posicion actualizada (6,7)->(6,8) - Liberando Mutex.
84:[JUGADOR 05] Movimiento derecha (2,1)->(3,1) - Solicitando Mutex casilla destino...
85:[JUGADOR 05] Mutex casilla adquirido - Verificando casilla (3,1)...
86:[JUGADOR 05] Posicion actualizada (2,1)->(3,1) - Liberando Mutex.
87:[JUGADOR 06] Movimiento derecha (4,8)->(5,8) - Solicitando Mutex casilla destino...
88:[JUGADOR 06] Mutex casilla adquirido - Verificando casilla (5,8)...
89:[JUGADOR 06] Posicion actualizada (4,8)->(5,8) - Liberando Mutex.
90:[JUGADOR 08] Movimiento derecha (5,7)->(6,7) - Solicitando Mutex casilla destino...
91:[JUGADOR 08] Mutex casilla adquirido - Verificando casilla (6,7)...
92:[JUGADOR 08] Posicion actualizada (5,7)->(6,7) - Liberando Mutex.
100:[JUGADOR 03] Movimiento arriba (1,1)->(1,0) - Solicitando Mutex casilla destino...
101:[JUGADOR 03] Mutex casilla adquirido - Verificando casilla (1,0)...
102:[JUGADOR 03] Posicion actualizada (1,1)->(1,0) - Liberando Mutex.
121:[JUGADOR 05] Movimiento abajo (3,1)->(3,2) - Solicitando Mutex casilla destino...
122:[JUGADOR 05] Mutex casilla adquirido - Verificando casilla (3,2)...
123:[JUGADOR 05] Posicion actualizada (3,1)->(3,2) - Liberando Mutex.
124:[JUGADOR 06] Movimiento derecha (5,8)->(6,8) - Solicitando Mutex casilla destino...
125:[JUGADOR 06] Mutex casilla adquirido - Verificando casilla (6,8)...
126:[JUGADOR 06] Casilla (6,8) ocupada - Liberando Mutex. Movimiento cancelado.
127:[JUGADOR 03] Movimiento abajo (1,0)->(1,1) - Solicitando Mutex casilla destino...
128:[JUGADOR 03] Mutex casilla adquirido - Verificando casilla (1,1)...
129:[JUGADOR 03] Posicion actualizada (1,0)->(1,1) - Liberando Mutex.

```

Figura 3: Movimiento concurrente de jugadores en el mapa. Cada jugador solicita el mutex de la casilla destino antes de modificar la matriz compartida, evitando inconsistencias en la posición de los jugadores.

Recolección de ítems

Otro resultado observable es la recolección de ítems. Cuando un jugador llega a una casilla con un objeto, intenta recogerlo entrando a una sección crítica protegida por el mutex individual del ítem. En los logs se observa la espera por el mutex, la validación del objeto, la recolección y la liberación del recurso.

```

224:[JUGADOR 03] Intentando recoger 'Pocion' en (4,2) - Esperando Mutex del ítem...
225:[JUGADOR 03] Mutex del ítem adquirido - Validando existencia de 'Pocion'...
226:[JUGADOR 03] ** ÍTEM RECOGIDO ** 'Pocion' - Total ítems: 1
227:[JUGADOR 03] Acción terminada - Liberando Mutex del ítem.
240:[JUGADOR 03] Intentando recoger 'Pocion' en (5,2) - Esperando Mutex del ítem...
241:[JUGADOR 03] Mutex del ítem adquirido - Validando existencia de 'Pocion'...
242:[JUGADOR 03] ** ÍTEM RECOGIDO ** 'Pocion' - Total ítems: 2
243:[JUGADOR 03] Acción terminada - Liberando Mutex del ítem.
291:[JUGADOR 04] Intentando recoger 'Escudo' en (0,4) - Esperando Mutex del ítem...
292:[JUGADOR 04] Mutex del ítem adquirido - Validando existencia de 'Escudo'...
293:[JUGADOR 04] ** ÍTEM RECOGIDO ** 'Escudo' - Total ítems: 1
294:[JUGADOR 04] Acción terminada - Liberando Mutex del ítem.

```

Figura 4: Recolección de ítems protegida con mutex individual. El jugador entra a la sección crítica, valida la existencia del objeto y lo recoge de forma segura.

Chat global con variable de condición

El sistema también muestra el funcionamiento del chat global. El hilo de broadcast permanece dormido mientras no hay mensajes usando `pthread_cond_wait()`. Cuando un jugador envía un mensaje, se manda una señal y el hilo despierta para procesarlo. Esto demuestra que el hilo de chat no realiza espera activa.

```
34:[CHAT] pthread_cond_wait() - Hilo dormido, esperando signal...
54:[CHAT] Variable de condicion: DESPIERTO - Signal recibido!
55:[CHAT] Procesando mensaje del Jugador 07: "Adios, me desconecto"
56:[CHAT] Mensaje distribuido a 1 jugadores conectados.
57:[CHAT] pthread_cond_wait() - Hilo dormido, esperando signal...
95:[CHAT] Variable de condicion: DESPIERTO - Signal recibido!
96:[CHAT] Procesando mensaje del Jugador 07: "Alguien quiere hacer equipo?"
97:[CHAT] Mensaje distribuido a 4 jugadores conectados.
98:[CHAT] pthread_cond_wait() - Hilo dormido, esperando signal...
102:[CHAT] Variable de condicion: DESPIERTO - Signal recibido!
103:[CHAT] Procesando mensaje del Jugador 08: "Hola a todos!"
104:[CHAT] Mensaje distribuido a 4 jugadores conectados.
105:[CHAT] pthread_cond_wait() - Hilo dormido, esperando signal...
114:[CHAT] Variable de condicion: DESPIERTO - Signal recibido!
115:[CHAT] Procesando mensaje del Jugador 01: "Hola a todos!"
116:[CHAT] Mensaje distribuido a 4 jugadores conectados.
117:[CHAT] pthread_cond_wait() - Hilo dormido, esperando signal...
131:[CHAT] Variable de condicion: DESPIERTO - Signal recibido!
132:[CHAT] Procesando mensaje del Jugador 04: "Vamos al centro del mapa"
133:[CHAT] Mensaje distribuido a 4 jugadores conectados.
134:[CHAT] pthread_cond_wait() - Hilo dormido, esperando signal...
138:[CHAT] Variable de condicion: DESPIERTO - Signal recibido!
139:[CHAT] Procesando mensaje del Jugador 01: "Necesito ayuda"
140:[CHAT] Mensaje distribuido a 4 jugadores conectados.
141:[CHAT] pthread_cond_wait() - Hilo dormido, esperando signal...
182:[CHAT] Variable de condicion: DESPIERTO - Signal recibido!
183:[CHAT] Procesando mensaje del Jugador 08: "GG bien jugado"
184:[CHAT] Mensaje distribuido a 4 jugadores conectados.
185:[CHAT] pthread_cond_wait() - Hilo dormido, esperando signal...
198:[CHAT] Variable de condicion: DESPIERTO - Signal recibido!
199:[CHAT] Procesando mensaje del Jugador 07: "Adios, me desconecto"
200:[CHAT] Mensaje distribuido a 4 jugadores conectados.
201:[CHAT] pthread_cond_wait() - Hilo dormido, esperando signal...
```

Figura 5: Sincronización del chat global mediante variable de condición. El hilo de broadcast permanece dormido en `pthread_cond_wait()` y despierta cuando un jugador envía un mensaje.

Respawn de ítems

El hilo de entorno ejecuta el respawn de ítems cada cierto intervalo de tiempo. En la evidencia se observa que el *World Thread* calcula coordenadas, espera el mutex de la casilla y coloca un nuevo objeto solamente si la posición está disponible.

```

33:[WORLD] World Thread iniciado - Respawn cada 4 segundos
41:[JUGADOR 02] Posicion actualizada (0,1)->(0,2) - Liberando Mutex.
47:[JUGADOR 08] Posicion actualizada (8,2)->(8,1) - Liberando Mutex.
50:[JUGADOR 02] Posicion actualizada (0,2)->(0,3) - Liberando Mutex.
53:[JUGADOR 08] Posicion actualizada (8,1)->(8,0) - Liberando Mutex.
59:[JUGADOR 01] Posicion actualizada (2,0)->(2,1) - Liberando Mutex.
62:[JUGADOR 08] Posicion actualizada (8,0)->(8,1) - Liberando Mutex.
65:[JUGADOR 02] Posicion actualizada (0,3)->(0,2) - Liberando Mutex.
68:[JUGADOR 08] Posicion actualizada (8,1)->(9,1) - Liberando Mutex.
74:[JUGADOR 05] Posicion actualizada (4,2)->(3,2) - Liberando Mutex.
80:[JUGADOR 06] Posicion actualizada (7,7)->(7,6) - Liberando Mutex.
87:[JUGADOR 05] Mensaje enviado - Liberando Mutex de chat. Signal enviado.
94:[JUGADOR 08] Mensaje enviado - Liberando Mutex de chat. Signal enviado.
101:[JUGADOR 01] Posicion actualizada (2,1)->(1,1) - Liberando Mutex.
106:[JUGADOR 01] Mensaje enviado - Liberando Mutex de chat. Signal enviado.
113:[JUGADOR 06] Posicion actualizada (7,6)->(7,7) - Liberando Mutex.
116:[JUGADOR 02] Posicion actualizada (0,2)->(0,1) - Liberando Mutex.
119:[JUGADOR 01] Posicion actualizada (1,1)->(1,2) - Liberando Mutex.
122:[JUGADOR 05] Posicion actualizada (3,2)->(2,2) - Liberando Mutex.
125:[JUGADOR 08] Mensaje enviado - Liberando Mutex de chat. Signal enviado.
157:[JUGADOR 01] Mensaje enviado - Liberando Mutex de chat. Signal enviado.
164:[JUGADOR 08] Posicion actualizada (9,1)->(9,2) - Liberando Mutex.
167:[JUGADOR 06] Posicion actualizada (7,7)->(7,8) - Liberando Mutex.
170:[JUGADOR 05] Posicion actualizada (2,2)->(2,3) - Liberando Mutex.
173:[JUGADOR 01] Posicion actualizada (1,2)->(1,1) - Liberando Mutex.
176:[JUGADOR 08] Mensaje enviado - Liberando Mutex de chat. Signal enviado.
181:[WORLD] Tiempo de respawn alcanzado - Calculando coordenadas...
182:[WORLD] Intentando casilla (8,2) - Esperando Mutex casilla...
183:[WORLD] Mutex adquirido - 'Escudo+' colocado en (8,2) - Liberando Mutex.
186:[JUGADOR 06] Mensaje enviado - Liberando Mutex de chat. Signal enviado.
193:[JUGADOR 01] Posicion actualizada (1,1)->(1,2) - Liberando Mutex.
196:[JUGADOR 02] Posicion actualizada (0,1)->(1,1) - Liberando Mutex.
199:[JUGADOR 05] Posicion actualizada (2,3)->(1,3) - Liberando Mutex.
202:[JUGADOR 01] Casilla (1,3) ocupada - Liberando Mutex. Movimiento cancelado.
205:[JUGADOR 08] Mensaje enviado - Liberando Mutex de chat. Signal enviado.

```

Figura 6: Hilo de entorno ejecutando el respawn de ítems. El World Thread calcula coordenadas, espera el mutex de la casilla y coloca un nuevo objeto solo si la posición está disponible.

Resumen final de la simulación

Al finalizar la simulación, el programa imprime un resumen final con métricas como intentos de conexión, conexiones rechazadas o en espera, movimientos totales, ítems recogidos, mensajes de chat y condiciones de carrera evitadas.

```

[SERVER] Todos los hilos de jugadores terminados.
[SERVER] Esperando terminacion de hilos de entorno...
[WORLD] World Thread terminado.
[SERVER] Hilos de entorno terminados.
[SERVER] Hilo de chat terminado.
[SERVER] Hilo de game loop terminado.

```

RESUMEN FINAL DE SIMULACION	
Intentos de conexion:	8
Conexiones rechazadas (espera):	3
Movimientos totales:	62
Items recogidos:	2
Mensajes de chat:	26
Race conditions evitadas:	0

JUGADOR	ITEMS	ESTADO
JUG 01	0	Desconectado
JUG 02	0	Desconectado
JUG 03	0	Desconectado
JUG 04	1	Desconectado
JUG 05	1	Desconectado
JUG 06	0	Desconectado
JUG 07	0	Desconectado
JUG 08	0	Desconectado

```

[SERVER] Fase 4: Destruyendo primitivas de sincronizacion...
[CLEANUP] Todas las primitivas destruidas. Memoria liberada.
[SERVER] Simulacion finalizada correctamente. Sin memory leaks.

```

Figura 7: Tabla de resumen final con métricas y tabla con datos de cada jugador al final.

6. Interpretación técnica

El proyecto logra demostrar varios conceptos importantes de Sistemas Operativos relacionados con concurrencia y sincronización.

Primero, se usan de hilos mediante funciones como `pthread_create()` y `pthread_join()`. Gracias a esto, el programa principal puede coordinar la ejecución de los jugadores, el entorno, el sistema de chat y el game loop al mismo tiempo.

También se usa la exclusión mutua usando `pthread_mutex_lock()` y `pthread_mutex_unlock()`. Los mutex se utilizan para proteger recursos compartidos como el mapa, los ítems, las estadísticas, el chat e incluso la salida en consola. Esto evita que varios hilos intenten modificar la misma información al mismo tiempo y provoquen errores.

Otro punto importante es el uso de semáforos mediante `sem_wait()` y `sem_post()`. En este caso, el semáforo representa los espacios disponibles dentro del servidor. Cuando el límite de jugadores se alcanza, los nuevos hilos deben esperar hasta que otro jugador salga y libere espacio.

El sistema de chat también enseña sincronización por eventos usando `pthread_cond_wait()` y `pthread_cond_signal()`. Esto permite que el hilo encargado del chat permanezca dormido hasta que aparezca un mensaje nuevo, evitando consumir CPU revisando constantemente si existe actividad.

Cada recurso compartido tiene su propia estrategia de sincronización. Por ejemplo, el mapa utiliza mutex por casilla, los ítems tienen mutex individuales, el chat trabaja con mutex y variables de condición, y el control de acceso del servidor se maneja con semáforos. Esto hace que la simulación sea más organizada y también más cercana a situaciones reales.

7. Limitaciones y trabajo futuro

El sistema cumple con el objetivo de demostrar la concurrencia y la sincronización, aunque todavía tiene algunas limitaciones.

Primero, la simulación no tiene una interfaz gráfica. Todo se ve por medio de logs en consola. Esto sirve para entender cómo funciona el sistema y ver lo que hacen los hilos, pero al final la representación visual del mundo queda muy simple.

También, los jugadores funcionan de manera automática. El usuario no puede controlar directamente a un jugador porque todas las acciones se generan al azar para provocar movimiento, mensajes en el chat y recolección de objetos.

Otro detalle es que el sistema no usa sockets reales. Todos los jugadores son hilos ejecutándose dentro del mismo proceso. Esto facilita trabajar los problemas de sincronización y concurrencia, pero no representa completamente cómo funcionaría un sistema cliente-servidor de verdad.

Como trabajo futuro, el proyecto podría mejorarse agregando sockets reales para que cada jugador funcione como un cliente independiente. Con esto la simulación se sentiría más cercana a un videojuego multijugador real.

Otra mejora interesante sería crear un modo sin sincronización para comparar directamente qué pasa antes y después de usar mutex, semáforos y variables de condición. Así se podrían notar más fácil los errores de concurrencia y los problemas que aparecen a cada rato cuando varios hilos comparten recursos.

También se podría agregar un sistema de intercambio de objetos entre jugadores. Esto ayudaría a representar problemas como el deadlock de una manera mucho más clara y fácil de entender. Finalmente, otra buena mejora sería implementar una interfaz visual sencilla donde se pueda observar el mapa y los movimientos de los jugadores en tiempo real.